Part 23: Feature Selection and Temporal Validation

Financial Data Science II Updated March 17, 2026

Part 23: Feature Selection and Temporal Validation

Part 22 considered the construction of candidate predictors. The next task is to determine which of those predictors merit further attention.

This is not a mechanical step. A feature may appear useful in sample and fail out of sample. A feature may also be statistically related to the target and yet be infeasible in real time.

For that reason, feature selection is best viewed as a sequence of screens rather than as a single algorithm.

Exercise. Explain the difference between the following statements.

1. A feature is correlated with the target in sample.
2. A feature is useful in a predictive model.

Basic Terms

We use the following terms in this Part.

A **screening rule** is a preliminary rule used to reduce a large candidate set. A **selection rule** determines which features are retained for modelling. **Redundancy** refers to overlap in the information carried by different features. **Stability** refers to whether a feature behaves similarly across periods or regimes. An **ablation study** compares a baseline specification to an expanded one in order to assess the incremental value of a feature or feature family.

Finally, we distinguish between a **ranking statistic** and a **modelling objective**. For example, a feature may be ranked by a univariate F-score or by mutual information, while the final predictive model is judged by out-of-sample RMSE, R^2, AUC, or some other metric.

Exercise. Why is it possible for two features to rank highly on their own but add little value when used together in the same model?

Why Feature Selection is Needed

Feature selection matters for at least four reasons.

1. Large feature sets can lead to unstable fitted models.
2. Many features are redundant transformations of the same underlying signal.
3. Financial relationships vary over time, so a feature that works in one period may fail in another.
4. Some features are costly to maintain or impossible to compute in a production setting.

The goal is not to maximize the number of features. The goal is to retain the subset that is informative, stable, and usable.

A Selection Framework

A practical sequence is the following.

1. Remove features that are infeasible or contaminated by leakage.
2. Remove features with too many missing values or too little variation.
3. Rank the remaining features using simple univariate measures.
4. Check whether the ranking is stable across time.
5. Evaluate incremental predictive value using walk-forward validation.

Univariate Screening

In this Part we use two simple ranking statistics.

1. **F-score:** emphasizes linear association with a continuous target.
2. **Mutual information:** allows for more general dependence.

A convenient summary is the **combined rank**, defined as the average of the rank under each criterion. This is a screening device, not a final objective function.

Example A: Daily Feature Screening

We return to the daily equity setting from Part 22. For illustration, we construct a modest feature set from prices and rough accounting ratios. We then rank the features and compare predictive performance as the feature set expands.

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd

from sklearn.feature_selection import f_regression, mutual_info_regression
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import balanced_accuracy_score, mean_squared_error, r2_score, roc_auc_s
from sklearn.model_selection import TimeSeriesSplit

from lecture_utils import (
    build_basic_daily_features,
    build_fundamental_features,
    build_lob_bars,
    build_lob_event_features,
    download_price_history,
    load_lobster_sample,
    locate_lobster_sample,
)
```

```
TICKER = 'AAPL'
START = '2016-01-01'
END = '2026-03-01'

px = download_price_history(TICKER, START, END)
daily = build_basic_daily_features(px, windows=(5, 21, 63))

fund = build_fundamental_features(TICKER, px.index, px['adj_close'])
if {'pe', 'pb'}.intersection(fund.columns):
    daily = daily.join(fund[[c for c in ['pe', 'pb'] if c in fund.columns]], how='left')

daily['target_fwd_ret_5d'] = px['adj_close'].shift(-5) / px['adj_close'] - 1
data_daily = daily.replace([np.inf, -np.inf], np.nan).dropna().copy()

X_daily = data_daily.drop(columns=['target_fwd_ret_5d'])
y_daily = data_daily['target_fwd_ret_5d']
X_daily.head()
```

	ret_1d	log_ret_1d	mom_5d	vol_5d	mom_21d	vol_21d	mom_63d	vol_63
Date								
2024-10-01	-0.029142	-0.029575	-0.005102	0.018809	-0.012183	0.015860	0.028156	0.0147
2024-10-02	0.002520	0.002517	0.001811	0.018739	0.018001	0.014633	0.024791	0.0147
2024-10-03	-0.004895	-0.004907	-0.008131	0.018661	0.021825	0.014533	-0.001806	0.0145
2024-10-04	0.005007	0.004995	-0.004346	0.018875	0.019876	0.014501	-0.003325	0.0144
2024-10-07	-0.022531	-0.022789	-0.048541	0.015258	0.003940	0.015305	-0.029445	0.0147

```
f_scores, _ = f_regression(X_daily, y_daily)
mi_scores = mutual_info_regression(X_daily, y_daily, random_state=42)

ranking_daily = pd.DataFrame({
    'feature': X_daily.columns,
    'f_score': f_scores,
    'mi_score': mi_scores,
})
ranking_daily['rank_f'] = ranking_daily['f_score'].rank(ascending=False, method='average')
ranking_daily['rank_mi'] = ranking_daily['mi_score'].rank(ascending=False, method='average')
ranking_daily['combined_rank'] = (ranking_daily['rank_f'] + ranking_daily['rank_mi']) / 2
ranking_daily = ranking_daily.sort_values('combined_rank').reset_index(drop=True)
ranking_daily
```

	feature	f_score	mi_score	rank_f	rank_mi	combined_rank
0	pb	28.372708	0.259856	1.0	2.0	1.5
1	pe	12.422412	0.271027	3.0	1.0	2.0
2	drawdown	7.921999	0.212520	4.0	3.0	3.5
3	mom_21d	7.029405	0.091762	5.0	5.0	5.0
4	mom_5d	18.259156	0.000000	2.0	11.0	6.5
5	vol_21d	0.676200	0.144032	9.0	4.0	6.5
6	vol_5d	3.515585	0.079588	6.0	8.0	7.0
7	ret_1d	0.934221	0.062242	8.0	9.0	8.5
8	log_ret_1d	0.951112	0.061004	7.0	10.0	8.5
9	mom_63d	0.003459	0.091008	11.0	6.0	8.5
10	vol_63d	0.592641	0.089446	10.0	7.0	8.5

The table above is a screening table. It is useful because it reduces the candidate set before more expensive validation. It is limited because it ignores interactions, redundancy, and changes across time.

```
def walk_forward_regression(X, y, model, n_splits=5):
    tscv = TimeSeriesSplit(n_splits=n_splits)
    rows = []
    for fold, (train_idx, test_idx) in enumerate(tscv.split(X), start=1):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        rows.append({
            'fold': fold,
            'rmse': mean_squared_error(y_test, pred) ** 0.5,
            'r2': r2_score(y_test, pred),
            'sign_accuracy': (np.sign(pred) == np.sign(y_test)).mean(),
        })
    return pd.DataFrame(rows)


top_counts = [3, 5, min(8, X_daily.shape[1]), X_daily.shape[1]]
top_counts = sorted(set(top_counts))
selection_summary = []

for k in top_counts:
    chosen = ranking_daily['feature'].iloc[:k].tolist()
    scores = walk_forward_regression(X_daily[chosen], y_daily, Ridge(alpha=1.0))
    selection_summary.append({
        'feature_set': f'top_{k}',
        'rmse': scores['rmse'].mean(),
        'r2': scores['r2'].mean(),
        'sign_accuracy': scores['sign_accuracy'].mean(),
    })

pd.DataFrame(selection_summary).sort_values('rmse').reset_index(drop=True)
```

	feature_set	rmse	r2	sign_accuracy
0	top_5	0.046920	-0.308405	0.473077
1	top_8	0.046929	-0.308613	0.473077
2	top_3	0.047044	-0.318992	0.484615
3	top_11	0.049712	-0.525411	0.480769

Exercise. Why might the feature set with the best univariate ranking fail to produce the best multivariate model?

Stability Across Time

A feature that ranks well over the full sample may do so only because of one favorable period. It is therefore useful to check whether a feature retains its rank across rolling subsamples.

```
windows = []
split_points = np.array_split(np.arange(len(X_daily)), 4)

for window_id, idx in enumerate(split_points, start=1):
    X_window = X_daily.iloc[idx]
    y_window = y_daily.iloc[idx]
    if len(X_window) < 50:
        continue
    f_scores_w, _ = f_regression(X_window, y_window)
    mi_scores_w = mutual_info_regression(X_window, y_window, random_state=42)
    temp = pd.DataFrame({
        'feature': X_daily.columns,
        'window': window_id,
        'f_score': f_scores_w,
        'mi_score': mi_scores_w,
    })
    temp['rank_f'] = temp['f_score'].rank(ascending=False, method='average')
    temp['rank_mi'] = temp['mi_score'].rank(ascending=False, method='average')
    temp['combined_rank'] = (temp['rank_f'] + temp['rank_mi']) / 2
    windows.append(temp[['feature', 'window', 'combined_rank']])

rank_stability = pd.concat(windows, ignore_index=True)
stability_summary = rank_stability.groupby('feature')['combined_rank'].agg(['mean', 'std']).
stability_summary
```

	mean	std
feature		
pe	2.5000	1.224745
drawdown	3.7500	0.957427
pb	3.7500	2.020726
vol_21d	5.2500	2.254625
vol_63d	5.6250	2.594064
mom_21d	5.6250	2.780138
vol_5d	5.8750	2.096624
mom_63d	6.3750	2.358495
mom_5d	7.1250	2.462214
log_ret_1d	9.9375	0.718070
ret_1d	10.1875	0.850857

Example B: LOB Feature Screening

We repeat the same logic in the LOB setting. The aim is not to build a definitive high-frequency model. The aim is to show how screening and ablation ideas apply to event-time data.

```
lobster_dir = locate_lobster_sample()
lob = load_lobster_sample(lobster_dir, levels=5)
lob_feat = build_lob_event_features(lob, levels=5)
bars = build_lob_bars(lob_feat)
bars['target_up_1s'] = (bars['mid'].shift(-1) > bars['mid']).astype(int)
bars = bars.dropna().copy()
bars.head()
```

	time_floor	time	bid_1	bid_size_1	ask_1	ask_size_1	mid	spread
0	34200	34200.918221	5857400	100	5858700	100	5858050.0	1300
1	34201	34201.840852	5854700	18	5857700	18	5856200.0	3000
2	34202	34202.756909	5854700	100	5856800	18	5855750.0	2100
3	34203	34203.954770	5854500	18	5857100	100	5855800.0	2600
4	34204	34204.894471	5855000	18	5856800	900	5855900.0	1800

```
feature_cols_lob = ['spread_bps', 'imbalance_l1', 'imbalance_l5', 'microprice_dev_bp', 'ofi_


def walk_forward_auc(feature_frame, y, feature_names, n_splits=5):
    tscv = TimeSeriesSplit(n_splits=n_splits)
    aucs = []
    baccs = []
    for train_idx, test_idx in tscv.split(feature_frame):
        X_train = feature_frame.iloc[train_idx][feature_names]
        X_test = feature_frame.iloc[test_idx][feature_names]
        y_train = y.iloc[train_idx]
        y_test = y.iloc[test_idx]
        model = LogisticRegression(max_iter=500)
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        prob = model.predict_proba(X_test)[:, 1]
        aucs.append(roc_auc_score(y_test, prob))
        baccs.append(balanced_accuracy_score(y_test, pred))
    return float(np.mean(aucs)), float(np.mean(baccs))


lob_univariate = []
```

```
y_lob = bars['target_up_1s']
for feature in feature_cols_lob:
    auc, bacc = walk_forward_auc(bars[feature_cols_lob], y_lob, [feature])
    lob_univariate.append({'feature': feature, 'auc': auc, 'balanced_accuracy': bacc})

lob_univariate = pd.DataFrame(lob_univariate).sort_values('auc', ascending=False).reset_inde
lob_univariate
```

	feature	auc	balanced_accuracy
0	msg_count	0.554703	0.501571
1	imbalance_l1	0.538607	0.500000
2	microprice_dev_bp	0.534970	0.500000
3	ofi_l1	0.524367	0.500130
4	spread_bps	0.506381	0.500000
5	imbalance_l5	0.502624	0.500000
6	ofi_l5	0.500580	0.500000
7	signed_size	0.494757	0.499957

```
feature_sets = {
    'base_book_state': ['spread_bps', 'imbalance_l1'],
    'book_plus_depth': ['spread_bps', 'imbalance_l1', 'imbalance_l5', 'microprice_dev_bp'],
    'book_plus_ofi': ['spread_bps', 'imbalance_l1', 'imbalance_l5', 'microprice_dev_bp', 'of
    'full_set': feature_cols_lob,
}

lob_set_summary = []
for name, cols in feature_sets.items():
    auc, bacc = walk_forward_auc(bars[feature_cols_lob], y_lob, cols)
    lob_set_summary.append({'feature_set': name, 'auc': auc, 'balanced_accuracy': bacc})

pd.DataFrame(lob_set_summary).sort_values('auc', ascending=False).reset_index(drop=True)
```

	feature_set	auc	balanced_accuracy
0	full_set	0.558731	0.503890
1	book_plus_ofi	0.539412	0.500062
2	book_plus_depth	0.534492	0.500000
3	base_book_state	0.528238	0.500000

The LOB example makes the logic of ablation particularly clear. One may begin with a minimal description of the book state, then add depth, then add OFI, and ask whether each expansion improves out-of-sample performance.

What Should Be Reported?

When presenting a selected feature set, it is useful to report at least the following.

1. The original candidate features.
2. The screening criteria.
3. The final selected subset.
4. The validation scheme.
5. The metric by which performance is judged.
6. The incremental contribution of the selected features relative to a simpler baseline.

This makes it easier to distinguish between a feature that is merely interesting and a feature that is useful for a forecasting problem.

Exercises

Exercise 1. Give an example of a feature that ranks well by mutual information but might still be rejected for practical reasons.

Exercise 2. Suppose that two features have nearly identical combined rank values. What additional information would you want before choosing between them?

Exercise 3. In the LOB setting, why is it useful to compare a base book-state model to a base-plus-OFI model rather than jumping immediately to the full feature set?

Exercise 4. Explain why a feature-selection rule based on a shuffled train/test split is not appropriate in this setting.

References

1. Cont, Kukanov, and Stoikov (2013), *The Price Impact of Order Book Events*.
2. Guyon and Elisseeff (2003), *An Introduction to Variable and Feature Selection*.
3. Lopez de Prado (2018), *Advances in Financial Machine Learning*.
4. LOBSTER documentation and sample files.